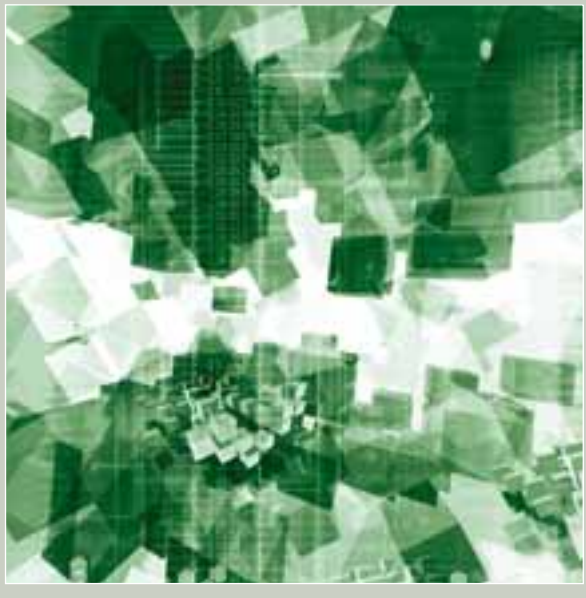
The Open University

M263 Unit 4
UNDERGRADUATE ICT AND COMPUTING

Building Blocks of Software



Introduction to classes

Unit 4

The Open University

M263 Building Blocks of Software

Unit 4

Introduction to classes

Study guide

In terms of their importance, the four sections of this unit are of about the same weight. Sections 2 and 3 are rather shorter than Section 4. However, Subsection 4.5 concerns points of theory (as does the aside at the end of Section 1). You will not be assessed on this material. It is included to help you to relate your work in this course to your study in other courses, or to your experience of actual programming languages. We recommend that you read these asides, but do not worry if they contain details that you find difficult to follow.

As with Unit 3, there are some practical computing activities involving use of the WorkPad associated with this unit. These are given separately. We suggest that you spend about one-fifth of your study time for this unit on practical work. The general comments about how to approach the practical work given in the Study Guide for Unit 3 apply equally to the work for this unit. It can all be done together, after you have finished study of the unit text, or done after each section, or done in parallel with study of the unit. There are practical activities relating to all the work in all sections of the unit.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Edited, designed and typeset by The Open University, using the Open University TEX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2689 3

2.1

Contents

Introduction

In Unit 3, you met the idea of a data type, and some particular data types involving basic forms of data. In an application, you may want to use data types tailored to the application situation. In this unit, we will consider how we can construct such a new (user-defined) data type.

The construction of a new data type can be divided into two parts: specification and implementation. This division parallels that for a user-defined process. The first step is to provide an outside view of the new data type. This is the specification. The specification tells us all that we need to know in order to use the new data type: to trace code involving variables of that type, or to design implementations of functions using it. (The descriptions given of the data types in Unit 3 were, in effect, specifications.)

We also need to describe how the data type is to be implemented. The implementation can use only what is already available in code. To begin with, only the data types described in Unit 3 are already available. We refer to the data types in Unit 3 as **primitive** types. They are the foundations on which new types are constructed. Once we have implemented some new type, then that can also be used as part of "what is available" to build further types.

In some programming languages, such as Smalltalk, there are no primitive types visible to the programmer.

In the course code, any new type is a **class** type (usually shortened to "class"). The processes associated with a class type are called the class **methods** (rather than type functions). You will see that there are a number of differences between the methods of a class, and the sort of computing functions that we discussed in Units 2 and 3. One reason why these differences are needed is to enable classes to participate in what is called **extension**. Extension helps with efficient building of new classes from similar classes that have already been constructed. You will see examples of extension in Section 4 of this unit.

In this course we will take a restricted view of the idea of class. Not all programming languages and authors use the terminology associated with class in exactly the same way. Consequently, the way we use terminology will not necessarily correspond exactly with its use in other books that you may read. Our aim here is to introduce some fundamental ideas associated with classes. However if, for example, one considers efficient storage of variables of class type, or how code is compiled, then more complicated issues come up, and these are issues that we shall not address.

In Section 1, we will be concerned with a particular example illustrating the basic features of a class. The class in our example is called **Spot**, and involves a spot on a two-dimensional grid that can move up or down, and left or right. We will look first at how this class can be specified. The specification provides the information needed to trace code involving variables of type **Spot**.

In Section 2, we consider user-defined processes involving **Spot**. To implement such processes, one only needs the class specification, which provides all the information needed to determine the effect of the methods of the class.

We refer to a set giving the possible states of an object of a particular class as the **data set** of that class. So for **Spot**, the data set is the set of points on a two-dimensional grid. In Section 3, we describe how the class **Spot**

can be implemented. To do this, we first describe an implementation of the data set of the class. This is done using **attributes**. We then give an implementation of each of the class methods.

An important technique in defining classes is extension. Suppose that you want to define a new class that only changes an existing class in some small way. Then rather than implementing this new class from scratch, you may be able to use extension. In Section 4, we look at two introductory examples of extension involving the class **Spot**. The first example has additional methods associated with the class. The second example has additional structure (this gives the spot a colour).

It is not an aim of this course to deal with large software systems. However, to provide a background to the significance of the ideas introduced in this unit, it is useful to refer to an example of this sort. In Unit 1, we touched on software that might be used by a supermarket till. If we take a wider view of the software that might be used in a supermarket, there may be a number of places where data is stored and processed. For example, there might be a number of tills, a computer where cash records are brought together, and another computer where stock control information is kept. Let us assume that the tills are identical, in that they all handle data of the same sort and in the same ways; that is, the data stored by each till has a common form, and each till needs to be able to perform the same processes on data. It is then natural to imagine a data type associated with the tills. This **Till** class would capture what is identical about the tills. It needs to encompass the form of data stored by each till, and the processes associated with the till. The form of data might include, for example:

- which staff member is currently on the till;
- the current transaction;
- the total cash that should currently be in the till.

There may be a variety of processes associated with the till data, such as:

- updating the current transaction when another item of shopping is processed;
- after a transaction is completed, sending a message to the stock control computer to update its record of what stock should be on the shelves;
- recording a change of the staff member on the till;
- recording any cash taken from the till to be moved to a central safe, which would include updates both to the till record and to the central cash record.

Although they have common features, each till in a supermarket is a separate entity, of course. Each will need its own identifier. We say that each individual till is an **instance** of the class **Till**. By giving the tills a common data type, we will ensure that they store and process data in the same sort of way. This will save effort in designing the software, for one only needs to do the design work once, for all the tills. A class therefore acts as a blueprint, which allows us to create lots of instances of that class.

Each individual till (that is, each instance of the class **Till**) stores data that will vary. The data stored in an individual till at any given time is called the **state** of that till. The state of a till will change with time. The current state of a till will include a number of things, such as the state of the current transaction, the current till operator, and the total cash that should be in the till at the time. The current transaction has its own internal structure, say as a sequence of till items, each of which is either a barcoded item or a weighed item. When an item is scanned, the state of the till will change by changing the current transaction. When a transaction is paid for in cash,

the feature of the stored data giving the total cash in the till will change. When there is a change in the person operating the till, this event will be recorded at the till, so that the feature of the stored data giving the till operator is changed.

Data stored in the stock control computer may include: the stock that should currently be on the shelves; the stock that should currently be in storage; records of movements of stock, both from storage to shelves, and of deliveries into storage, together with a staff member responsible for each such movement. This data is quite different from that stored in a till, but there are some common elements to the two forms of data. For example, both involve some way of identifying each member of staff. Looking inside the complicated forms of data that may be used in the two cases — a till, and the stock record — it is likely to be sensible to handle common aspects of highly structured data in the same way.

So the data types required in an application may have a complicated structure, but such types can be built up from simpler elements. The method of implementing classes described in this unit enables one to construct, from the primitive types, the more elaborate classes that may be needed in an application. It can also be used to ensure appropriate common structure between different classes, for example by using the same staff member class in building both a till class and a stock control class.

1 Specifying the class Spot

The purpose of this section is to introduce a number of important, fundamental concepts relating to the idea of class. This is done through an example, the class **Spot**. A variable of type **Spot** can move around the points on a two-dimensional grid (see Figure 1.1). Here, we are not interested in how this class **Spot** might be used in an application. It is chosen as an example that illustrates a number of key points about classes.

It is usual to refer to an instance of a class (as opposed to a variable of primitive type) as an **object**. Possible states of an object of type **Spot** are illustrated in Figure 1.1(a). These are points on a two-dimensional grid. The methods of a class play the role of type functions in a data type. The class **Spot** has **methods** that enable an object to be moved around the grid. These methods are called *rightOne*, *leftOne*, *upOne* and *downOne*. Each of these methods moves an object of type **Spot** to a place on the grid adjacent to its current position. Suppose, for example, that *aSpot* is an object of type **Spot**, and *aSpot* is at the point shown in Figure 1.1(b). If *upOne* is applied to *aSpot*, then *aSpot* is moved to the point shown in Figure 1.1(c). Notice that applying *upOne* to *aSpot* changes the state of *aSpot*.

An object *aSpot* of type **Spot** is created as usual, using the statement:

var *aSpot* **in Spot**

(In this course we will simply say that the variable *aSpot* "is" an object. To reflect what goes on inside the computer's memory, it would be more precise to say that the variable *aSpot* **references** an object, because the variable *aSpot* will hold the memory location of the object rather than the object itself.)

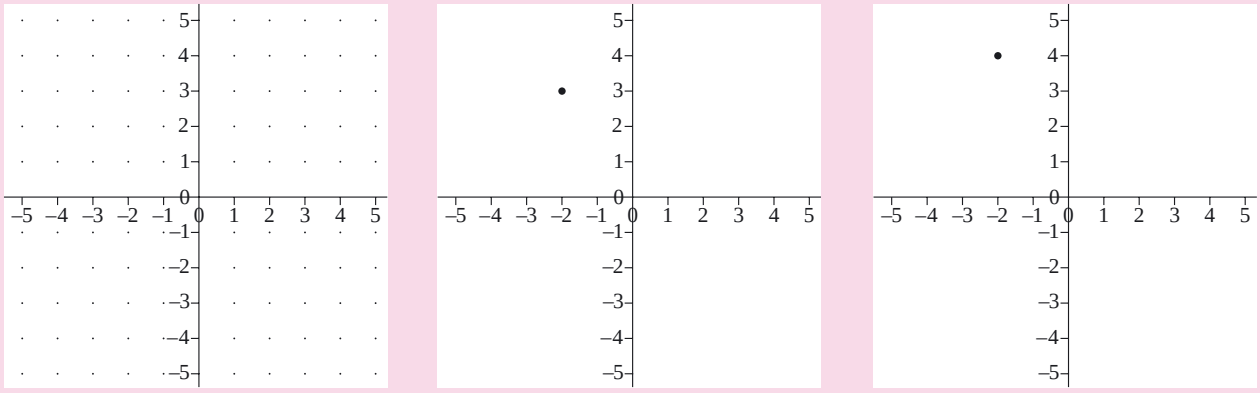


Figure 1.1 The possible states of an object *aSpot* of type **Spot** are shown in (a). If *upOne* is applied to *aSpot* when in the state shown in (b), then *aSpot* is moved to the state shown in (c).

The class **Spot** has a default state. Any object of type **Spot** that has just been created by a **var** declaration is in this default state.

We can introduce a notation to represent the possible states of an object of type **Spot** by expressing the position of the spot using a coordinate system set up as follows.

Take the default state as the origin.
Take the up direction as the y-axis.
Take the right direction as the x-axis.

The direction of the axes is an arbitrary decision.

This coordinate system is shown in Figure 1.1. The default state is then represented by the ordered pair $(0, 0)$. An application of *upOne* to a spot in the default state will take it to $(0, 1)$. An application of *rightOne* to a spot in the default state would take it to $(1, 0)$. An application of *downOne* to a spot in the default state would take it to $(0, -1)$. An application of *leftOne* to a spot in the default state would take it to $(-1, 0)$.

Activity 1.1

(a) In Figure 1.2, what is the state of an object of type **Spot** if it is at: (i) the point A; (ii) the point B; (iii) the point C?

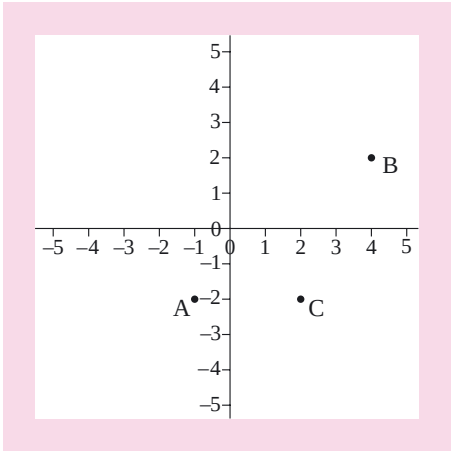


Figure 1.2

(b) Suppose that *aSpot* is in the state $(2, -1)$. What will be the state of *aSpot* after an application of *upOne*?

Discussion

(a) The states are:
(i) $(-1, -2)$; (ii) $(4, 2)$; (iii) $(2, -2)$.

(b) Applying *upOne* will increase, by 1, the second number giving the state of *aSpot*. The new state will be $(2, 0)$.

The application of a method to an object is written using a notation different from that used for calling a function. For example, to apply *upOne* to the object *aSpot*, we write

aSpot.upOne()

We will refer to this as the **dot notation**. Notice that there is no assignment involved in this statement. (We do not write $aSpot \leftarrow aSpot.upOne()$). The method *upOne* is not a function that returns a value.

In some programming languages, code statements of the form *aSpot.upOne*() are described as passing the method *upOne* as a **message** to the receiver, *aSpot*.

A specification of the method *upOne* is given below.

method *upOne*() **state change**
pre **true**.
post The spot is moved upwards on the grid by one position. (So if the state of the object is (X, Y) before *upOne* is applied, then after application it will be $(X, Y + 1)$.)

The functions that you met in earlier units all returned a value. The process *upOne* is of a different sort. Application of this method changes the state of a variable of type **Spot**. This is an important feature of *upOne*, and is stated in the first line of the specification, as part of the signature line. The signature line does not state a type for a returned value — since *upOne* does not return a value!

A second point to notice about this specification is that the type **Spot** does not appear in the signature line. We do not need to mention **Spot**, because a method of the class **Spot** can only be applied to an object of type **Spot**. Once you know that *upOne* is a method of the class **Spot**, you know that *upOne* can only be applied to an object of type **Spot** (with that application written using the dot notation).

You may notice one other thing about this specification: the empty brackets () after *upOne*. These tell you that *upOne* does not have any parameters. We will explain the idea of parameter later (Section 4), when we come to an example of a method that does have a parameter. These empty brackets are also used when the method is called in code.

Not all methods change the state of the object to which they are applied. Some methods do return a value. For example, the class **Spot** has a method *getXPos*. This method returns a value. It gives the x-coordinate, that is, the first number in the pair representing the position of the spot. For example, if the object *aSpot* is at $(3, 2)$, then *aSpot.getXPos*() returns the value 3.

Notice that application of any method is written using the dot notation. This notation is used both for methods that change state and for methods that return a value.

A specification of the method *getXPos* is given below.

method *getXPos*() **return in Int**
pre **true**.
post For a spot with state (X, Y), the returned value is X.

Many programming languages include methods that do two things: they change the state of the object to which they are applied *and* return a value. However, for simplicity, the course code does not include any such methods.

Activity 1.2

The method *getYPos* returns the second number in the pair giving the current state of an object of type **Spot**. Give a specification of *getYPos*.

Discussion A specification is given below. The specification is similar to that for *getXPos*, except that this time it is the second number in the pair giving the state that is returned.

method *getYPos*() **return in Int**
pre **true**.
post For a spot with state (X, Y), the returned value is Y.

Activity 1.3

The method *downOne* changes the state of an object of type **Spot** by moving it down one place on the grid. Give a specification of *downOne*.

Discussion A specification is given below. It is similar to that for *upOne*, except that this time the spot is moved down one place on the grid.

method *downOne*() **state change**
pre **true**.
post The spot is moved down on the grid by one position. (So if the state of the object is (X, Y) before *downOne* is applied, then it will be $(X, Y - 1)$ after application.)

A code fragment involving a variable of type **Spot** is given below. Note that the expression *spotOne.getXPos*() returns a value that is an integer. This can be stored in *num* using assignment.

```
var spotOne in Spot
var num in Int
spotOne.upOne()
spotOne.rightOne()
spotOne.rightOne()
num <-- spotOne.getXPos()
spotOne.rightOne()
```

A trace of the states of the variables *spotOne* and *num* as this code fragment is executed is given below. (Note that, if the state of *spotOne* is given by (X, Y), then an application of *upOne* will increase the second number, Y, by 1. An application of *rightOne* will increase the first number, X, by 1.)

	spotOne	*num*	
var *spotOne* **in Spot**	$(0, 0)$		$(0, 0)$ is the default state of **Spot**.
var *num* **in Int**	$(0, 0)$	-	**Int** does not have a default.
spotOne.upOne()	$(0, 1)$	-	
spotOne.rightOne()	$(1, 1)$	-	
spotOne.rightOne()	$(2, 1)$	-	
num ← *spotOne.getXPos*()	$(2, 1)$	2	
spotOne.rightOne()	$(3, 1)$	2	

Notice two things here. Application of the method *getXPos* to *spotOne* does not change the state of *spotOne*. (It just obtains information about the state of *spotOne* at the moment when the method is applied.) And the final application of *rightOne* to *spotOne* does not change *num*. The assignment *num* ← *spotOne*. *getXPos*() sets the state of *num* to the value returned by *spotOne.getXPos*() at that moment. It does *not* establish any permanent link between *spotOne* and *num*. The state of *num* will change only if we make another assignment to *num*.

When doing exercises involving the class **Spot**, to save space we will usually give traces using just the coordinates of the spot.

The WorkPad will show the effect of code involving **Spot** graphically, and in your own work you may find that using sketch diagrams helps you to visualise the way the state of a spot variable changes.

Activity 1.4

Trace the states of the variables *spotOne* and *num* when the code fragment below is executed.

```
var spotOne in Spot
var num in Int
spotOne.upOne()
spotOne.leftOne()
spotOne.leftOne()
num <-- spotOne.getXPos()
spotOne.downOne()
spotOne.rightOne()
spotOne.rightOne()
num <-- spotOne.getXPos()
```

Discussion A trace is given below.

	spotOne	*num*	
var *spotOne* **in Spot**	$(0, 0)$		The default state for **Spot**.
var *num* **in Int**	$(0, 0)$	-	
spotOne.upOne()	$(0, 1)$	-	
spotOne.leftOne()	$(-1, 1)$	-	
spotOne.leftOne()	$(-2, 1)$	-	
num ← *spotOne.getXPos*()	$(-2, 1)$	-2	
spotOne.downOne()	$(-2, 0)$	-2	
spotOne.rightOne()	$(-1, 0)$	-2	
spotOne.rightOne()	$(0, 0)$	-2	
num ← *spotOne.getXPos*()	$(0, 0)$	0	

Where there is a default state for a class, it is sensible to include variable declarations in a trace. It is not usually necessary to include other variable declarations.

A specification of the class **Spot** is given below. This specification of the class gives you the information you need when tracing or writing code involving variables of type **Spot**. The specification gives the following information.

- The possible states of an object of type **Spot**.
- The notation that we will use for these states when tracing code involving variables of type **Spot**.
- The default state of the class. A variable of type **Spot** has this default state after being created by a **var** declaration.
- A list of the methods of the class.
- A specification of each of the methods.

Spot **Class specification**

States The state of an object *mySpot* is an ordered pair of integers that define the position of *mySpot* on a two-dimensional square grid.

State notation (X, Y) means that the spot is X places to the right of the default, and Y places up from the default.

Default The spot is at $(0, 0)$.

Methods
getXPos() **return in Int**
getYPos() **return in Int**
upOne() **state change**
downOne() **state change**
leftOne() **state change**
rightOne() **state change**

To keep the boxed summary of the class brief, only the signature line is given for each method. The full method specifications are given after the box.

method *getXPos*() **return in Int**
pre **true**.
post For a spot with state (X, Y), the returned value is X.

method *getYPos*() **return in Int**
pre **true**.
post For a spot with state (X, Y), the returned value is Y.

method *upOne*() **state change**
pre **true**.
post The spot is moved upwards on the grid by one position.

method *downOne*() **state change**
pre **true**.
post The spot is moved downwards on the grid by one position.

method *leftOne*() **state change**
pre **true**.
post The spot is moved left on the grid by one position.

method *rightOne*() **state change**
pre **true**.
post The spot is moved right on the grid by one position.

The state notation (X, Y) is just a notation for use in text. We cannot use it as a literal value in code to set the state of an object. So, for example, *mySpot* $\leftarrow (1, 2)$ is not a valid statement in the course code. There are no literal values for the class **Spot**.

Activity 1.5

Trace the states of the variables as the code fragment below is executed.

```
var count in Int
var index in Int
var spotOne in Spot
count <-- 3
for (index <-- 1 to count)
{
  spotOne.rightOne()
}
for (index <-- 1 to count)
{
  spotOne.upOne()
}
```

Discussion A trace is given below. Notice that each of the **for** loops is executed with *index* set to: 1, 2 and 3.

Comment	*index*	*count*	*spotOne*
var *spotOne* **in Spot**	-	-	$(0, 0)$
count $\leftarrow 3$	-	3	$(0, 0)$
loop	1	3	$(1, 0)$
	2	3	$(2, 0)$
loop end	3	3	$(3, 0)$
second loop	1	3	$(3, 1)$
	2	3	$(3, 2)$
loop end	3	3	$(3, 3)$

Suppose that *theSpot* is an object of type **Spot**. The code fragment below will move the object *theSpot* up two places.

theSpot.*upOne*()
theSpot.*upOne*()

Now you might think that you could write *theSpot*.*upOne*().*upOne*() to achieve the same effect. However, this is not a valid statement in the course code. The statement *theSpot*.*upOne*() changes the state of the object *theSpot* (by moving it up one place). The expression *theSpot*.*upOne*(), however, does not represent the object *theSpot* in the changed state. That object still has the identifier *theSpot*, and the second application of the method *upOne* needs to be addressed directly to that object. In fact, *theSpot*.*upOne*() does not represent any object or value, because *upOne* does not return a value.

Aside

The following comments cover some rather subtle points. Do not expect to understand them at a first reading. They are here to help you to relate your work in this course to what you may encounter elsewhere. We look first at the mathematical function associated with a method. Let *States* be the set of all possible states of an object of type **Spot**.

Consider the method *getXPos*. This is addressed to an object of type **Spot**, and returns a value that is an integer. So this method can be shown to correspond to a mathematical function with the signature $States \rightarrow Int$. This mathematical function has a possible state of a spot as input, and returns a value in *Int*.

Now consider the method *upOne*. This method does not return a value. Rather, it changes the state of an object of type **Spot**. So this method can be shown to correspond to a mathematical function that has as input a possible state of a spot and whose output is also a possible state of a spot. So this method corresponds to a mathematical function with signature $States \rightarrow States$.

In the course code, a method will not change the state of any object other than the object to which it is addressed. What is more, any method in this course will either:

1. change the state of the object to which it is addressed (and return nothing), or
2. return a value, and leave the state of the object to which it is addressed unchanged.

A method of type 1 corresponds to a mathematical function with signature

$$States \rightarrow States$$

A method of type 2 corresponds to a mathematical function with signature

$$States \rightarrow Outputs$$

(where *Outputs* is a set containing the values that may be returned).

For example, for the method *getXPos*, *Outputs* is the set *Int*.

This separation of the two possible types of behaviour makes it much easier to trace the effect of methods in code.

In principle, a method might be of a more general form, with signature

$$States \rightarrow States \times Outputs$$

Such a method would have two separate effects: it would change the state of the object to which it was addressed, and would also return a value. However, none of the methods in this course will combine the two forms of behaviour.

Many programming languages do allow methods that combine both types of behaviour.

As you will see in Section 4, a method may have one or more parameters. These are additional inputs to the corresponding mathematical function. The most general form of method would correspond to a mathematical function with signature

$$States \times Inputs \rightarrow States \times Outputs$$

where *Inputs* is the set of all possible values of the parameters of the method.

The notation used to show the application of a method raises a couple of points. Consider for a moment a method *aMethod*() that both changes the state of the object to which it is addressed and returns a value. In such a case, the statement *anObject.aMethod*() firstly implies that the relevant change is made to the state of the object *anObject*. Also, however,

anObject.aMethod() represents the returned value. If this value is wanted, it needs to be captured (say by assigning it to a variable). Some programming languages have methods whose key role is to change the state of some object, but which also return a value. This value is usually not wanted. That is not a problem — you simply do not bother to capture the value. The use of such methods can be confusing, however, so we have chosen to avoid them in this course.

The course code uses the dot notation for application of all methods, both those that change a state and those that return a value. To know whether *anObject.aMethod*() means a value, or that the state of the variable *anObject* is to be changed, you need to look at the specification of *aMethod*.

End of Aside

Objectives for Section 1

After studying this section you should be able to do the following.

- ❒ Recognise and use the terminology: object, class, method, instance of a class.
- ❒ Interpret and use notation associated with the class **Spot**, including its methods.
- ❒ Be aware that a variable of type **Spot** is created in the default state.
- ❒ Use the specification of the class **Spot** to find the effect of methods of that class.
- ❒ Trace a code fragment involving variables of type **Spot**.
- ❒ Be aware that there are no literal values for the class **Spot** available in code.

Exercises on Section 1

Exercise 1.1

The state of the object *spotOne* is shown in Figure 1.3. Give the ordered pair representing the state of *spotOne*. If the statements below are then executed, what will be the resulting state of *spotOne*?

spotOne.downOne()
spotOne.rightOne()
spotOne.rightOne()

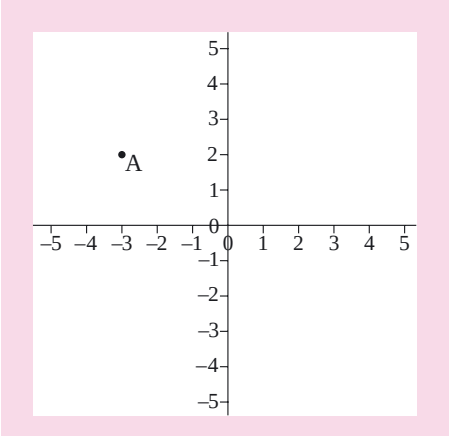


Figure 1.3

Exercise 1.2

Trace the states of the variables as the code fragment below is executed.

```
var count in Int
var index in Int
var spotOne in Spot
count <-- 1
for (index <-- 1 to count)
{
  spotOne.upOne()
}
count <-- count + 1
for (index <-- 1 to count)
{
  spotOne.rightOne()
}
count <-- count + 1
for (index <-- 1 to count)
{
  spotOne.downOne()
}
```

2 Code using the class Spot

In Section 1, you saw small fragments of code involving variables of type **Spot**. Here, we will look at more systematic code using **Spot**, and at functions involving this class. We also look at copying the state of a variable of class type. For variables of primitive type, that is achieved by assignment, but a slightly different approach has to be used for variables of class type.

2.1 Fragments of code that change the state of an object

A fragment of code is given below, and also a trace giving the states of the variables as that fragment is executed. Note that the final state of *aSpot* is $(0, 4)$.

```
var index in Int
var aSpot in Spot
for (index <-- 1 to 4)
{
   aSpot.upOne()
}
```

Comment	*index*	*aSpot*
var *aSpot* **in Spot**	-	$(0, 0)$
loop	1	$(0, 1)$
	2	$(0, 2)$
	3	$(0, 3)$
loop end	4	$(0, 4)$

Activity 2.1

In *Frag1* below, *aSpot* is a variable of type **Spot** and *index* and *count* are of type **Int**.

```
/* Frag1 */
for (index ← 1 to count)
{
    aSpot.upOne()
}
```

(a) Suppose that before the code in *Frag1* is executed, the state of *aSpot* is $(1, 2)$, and the state of *count* is 5. Trace *Frag1*. What is the state of *aSpot* after *Frag1* has been executed?

(b) Suppose that the code in *Frag1* is executed with both *count* and *aSpot* in some general state (where $count > 0$). Describe the effect of the code in *Frag1*.

Discussion

(a) A trace is given below. The final state of *aSpot* is (1,7).

Comment	*index*	*count*	*aSpot*
states from previous code	-	5	$(1, 2)$
loop	1	5	$(1, 3)$
	2	5	$(1, 4)$
	3	5	$(1, 5)$
	4	5	$(1, 6)$
loop end	5	5	$(1, 7)$

(b) In general, the effect of *Frag1* is to move *aSpot* up *count* places. So the state of *aSpot* changes from (X, Y), say, to $(X, Y + count)$.

Activity 2.2

Write a code fragment that has the effect of changing the state of an object *aSpot* of type **Spot** from (X, Y) to $(X - count, Y)$ (where $count > 0$).

Discussion The only methods available to *aSpot* are those that move *aSpot* one place at a time, either up, down, left or right. To decrease the x-coordinate, we have to apply *leftOne* repeatedly. The code fragment below has the desired effect.

```
for (index ← 1 to count)
{
    aSpot.leftOne()
}
```

2.2 Assignment for objects

If a and b are variables of the same class type, then the statement

$a \leftarrow b.clone()$

means "change the state of the object a to become identical to the current state of the object b". This is done in such a way that a and b remain independent objects having distinct identities. Changing the state of one of them at a later stage has no effect on the state of the other. This is exactly the same meaning as the assignment $a \leftarrow b$ for variables a and b of primitive type. That also does not create a lasting link between the values of a and b. For a and b of primitive type, a and b will have the same value just after the assignment $a \leftarrow b$. But if the value of one of a or b is changed later on, this will have no effect on the other one.

The course code does not use assignment of the form $a \leftarrow b$ for variables of class type. This is because in many programming languages an assignment $a \leftarrow b$, where a and b are objects, will result in a permanent link between the variables a and b.

We shall assume that all classes permit assignment for objects in the form $a \leftarrow b.clone()$. For a statement $a \leftarrow b.clone()$ to be valid, the objects a and b must be of the same type.

This is not the case in all programming languages.

Aside

The reasons for using *clone* for copying the state of a variable of class type relate to issues that we shall not discuss in this course. It is worth mentioning them briefly, however, to help you to relate your work here to reading that you may do elsewhere. If we take into account the internal structure of objects, and efficient approaches to memory management when code is executed, then a number of issues arise. These considerations lead to object-oriented programming languages having different forms of copying an object b to an object a. These include: linking b to a by a reference so that b and a reference precisely the same object; copying b to a in such a way that the two objects share some of their internal structure; and forming in b a completely separate and independent copy of a (known as **deep copy**).

The last of these, deep copy, is the only form of copying for objects that we shall consider in this course. When code is executed, forming a deep copy takes more time and uses more memory than simply referencing another object. However, the effect of deep copy is easier to understand. Other forms

of copying link variables. So, for objects a and b, after a statement $a \leftarrow b$ in many programming languages, later changes to the state of b will have the same effect on the state of a. The course code avoids forming permanent links between variables by insisting on the use of *clone* in assignments where objects are concerned. In our code, after the state of a variable is copied, either by an assignment $a \leftarrow b$ (for variables of primitive type) or else by $a \leftarrow b.clone()$ (for variables of class type), later changes to the state of b have no effect on the state of a (and vice versa).

End of Aside

Recall, see page 12, that there are no literals available for the class **Spot**. For a variable *aSpot* of type **Spot**, a statement such as $aSpot \leftarrow (2, 3).clone()$ would not be valid code. (Nor would a statement of the form $aSpot \leftarrow (2, 3)$ be valid.) The notation $(2, 3)$ is only used for showing states in traces — it is not a notation for literals of type **Spot** in code.

Activity 2.3

Suppose that *spotOne* and *spotTwo* are objects of type **Spot**. Which of the following statements is valid in the course code?

(a) $spotOne \leftarrow spotTwo$

(b) $spotOne \leftarrow spotTwo.clone()$

(c) $spotOne \leftarrow (5, 4).clone()$

Discussion

(a) For a class type such as **Spot**, the course code does not use assignment without the use of *clone*.

(b) This is valid.

(c) $(5, 4)$ is a notation we use to represent a possible state of an object of type **Spot**, but it is not a literal value of type **Spot**. The notation $(5, 4)$ cannot be used in code.

If a function returns a value that is a spot, then there is no need to use *clone* when assigning the value returned by that function to an object. For example, the function *ABOVE* discussed in Activity 2.5 below returns a spot. Suppose that *aSpot* is a variable of type **Spot**. Then we can assign a value returned above by *ABOVE* to *aSpot* by a statement such as this.

$aSpot \leftarrow ABOVE(7)$

A statement using *clone* would also be valid, but use of *clone* is not necessary here. The object $ABOVE(7)$ returned by a function is transient — there is no danger of the statement above forming a permanent link between two variables.

If a class type has literals available, there is again no need to use *clone* when assigning a literal value to an object.

Activity 2.4

Trace the states of the variables as the code fragment below is executed.

```
var index in Int
var spotOne in Spot
var spotTwo in Spot
var spotThree in Spot
for (index <-- 1 to 3)
{
  spotOne.rightOne()
}
spotTwo <-- spotOne.clone()
spotThree <-- spotOne.clone()
for (index <-- 1 to 3)
{
  spotOne.leftOne()
  spotTwo.upOne()
  spotThree.downOne()
}
```

Discussion A trace is given below.

Comment	*index*	*spotOne*	*spotTwo*	*spotThree*
var *spotOne* **in Spot**	-	$(0,0)$		
var *spotTwo* **in Spot**	-	$(0,0)$	$(0,0)$	
var *spotThree* **in Spot**	-	$(0,0)$	$(0,0)$	$(0,0)$
loop	1	$(1,0)$	$(0,0)$	$(0,0)$
	2	$(2,0)$	$(0,0)$	$(0,0)$
loop end	3	$(3,0)$	$(0,0)$	$(0,0)$
$spotTwo \leftarrow spotOne.clone()$	-	$(3,0)$	$(3,0)$	$(0,0)$
$spotThree \leftarrow spotOne.clone()$	-	$(3,0)$	$(3,0)$	$(3,0)$
loop	1	$(2,0)$	$(3,1)$	$(3,-1)$
	2	$(1,0)$	$(3,2)$	$(3,-2)$
loop end	3	$(0,0)$	$(3,3)$	$(3,-3)$

2.3 Functions involving the class Spot

In Units 2 and 3 you saw examples of the specification and implementation of user-defined functions (that is, functions that are not type functions of a data type). So far, you have only seen functions whose signature line is given in terms of primitive types. Class types are also permitted in the signature line of a user-defined function, and, in particular, the type **Spot** may appear there.

Activity 2.5

Give a specification and implementation of a function *ABOVE* that inputs an integer $i > 0$ and returns a spot in the state $(0, i)$.

Discussion For the implementation, we need to declare a local variable of type **Spot**, which will be in the default state, $(0, 0)$. We then need to move it up i places.

function *ABOVE*(i **in Int**) **return in Spot**
pre $i > 0$.
post The returned value is a spot with state $(0, i)$.

```
function ABOVE(i)
{
var aSpot in Spot
var index in Int
for (index <-- 1 to i)
  {aSpot.upOne()}
return aSpot
}
```

EXAMPLE 2.1

Below, we give a specification and implementation of a function *GOLEFT* that inputs a spot (s) and an integer (n), and returns a spot n places to the left of s.

function *GOLEFT*(s **in Spot**, n **in Int**) **return in Spot**
pre $n \geq 0$.
post The returned value is a spot n places to the left of s.

```
function GOLEFT(s,n)
{
var aSpot in Spot
var index in Int
aSpot <-- s.clone()
for (index <-- 1 to n)
  {aSpot.leftOne()}
return aSpot
}
```

Example 2.1 illustrates certain features of the course code that are not shared by all programming languages. Firstly, a user-defined function must not change the state of any of its inputs. As explained in Unit 2, in code to implement a function, a variable identifying an input must not have its state changed within the function.

The point that a user-defined function must not change the state of any of its inputs needs particular attention when the inputs are of class type. We must take care not to include in the function code any statement that applies a state-changing method to an input variable, since the effect of that would be to change the state of that input variable. This is why we need to include the local variable *aSpot* in the implementation in Example 2.1. The code below would not be a valid implementation of *GOLEFT* in the course code. (Code of the form below will work in many programming languages,

and will have the side-effect of changing the state of the object of type **Spot** that is used as the input in a call of the function.)

function *GOLEFT*(s, n)
{
var *index* **in Int**
for (*index* $\leftarrow$1 **to** n)
    {*s.leftOne*()}
return s
}

Activity 2.6

Give an implementation of the function *SAMEPLACE*, specified below.

function *SAMEPLACE*(*spotA* **in Spot**, *spotB* **in Spot**) **return in Bool**
pre **true**.
post The returned value is **true** if *spotA* and *spotB* are at the same place on the grid and is **false** otherwise.

Discussion Two spots are at the same place on the grid only if the x-coordinates of the two spots are equal and the y-coordinates of the two spots are also equal. The x-coordinate can be retrieved using the method *getXPos*, and the y-coordinate using *getYPos*. We give an implementation of *SAMEPLACE* below, using == to test for equality, first of the x-coordinates, and then of the y-coordinates.

```
function SAMEPLACE(spotA, spotB)
{
var b1 in Bool
var b2 in Bool
b1 <-- (spotA.getXPos() == spotB.getXPos())
/* b1 is true if spotA and spotB have the
 same x-coordinate. */
b2 <-- (spotA.getYPos() == spotB.getYPos())
/* b2 is true if spotA and spotB have the
 same y-coordinate. */
return (b1 /\ b2)
}
```

Recall that the Boolean operation "and" has infix notation $\wedge$.

Notice that in the implementation in Activity 2.6, it is not necessary to introduce local variables of **Spot** type, and copy *spotA* and *spotB* to them. That is because application of the methods *getXPos* and *getYPos* to *spotA* and *spotB* does not change the state of *spotA* and *spotB*. Each of these methods returns a value, and does not change the state of the object to which it is applied.

Objectives for Section 2

After studying this section you should be able to do the following.

- ❒ Write code involving variables of type **Spot** to achieve a specified purpose, either as a function, or as a fragment to achieve a desired state change.
- ❒ Be aware that, for variables of class type, the statement $a \leftarrow b.clone()$ (and not $a \leftarrow b$) is used in the course code to change the state of a to become identical to that of b.

Exercises on Section 2

Exercise 2.1

(a) Give a code fragment that has the effect of changing the state of an object *aSpot* of type **Spot** by moving it down n places.

(b) Give a specification and implementation of a function *GODOWN* that inputs a spot (s) and an integer ($n \geq 0$), and returns a spot n places below s.

Exercise 2.2

(a) Give an implementation of the function *U1R2* specified below.

function *U1R2*(s **in Spot**) **return in Spot**
pre **true**.
post The returned value is a spot one place above and two places to the right of s. (So, for example, if s has state (5,2) then the returned value is a spot at (7,3).)

(b) With *U1R2* as specified in (a), trace the states of the variables as the code below is executed.

```
var spotOne in Spot
var spotTwo in Spot
var spotThree in Spot
spotTwo <-- U1R2(spotOne)
spotThree <-- U1R2(spotTwo)
```

Exercise 2.3

In Figure 2.1, point B is above and to the right of point A. Points A, B, C and D form the corners of a rectangle. So if A has coordinates (X, Y) and B has coordinates (P, Q) then C has coordinates (X, Q) and D has coordinates (P, Y).

(a) Suppose that *spotOne* and *spotTwo* are objects of type **Spot**, with *spotOne* at A and *spotTwo* at B. Using methods of **Spot** (and primitive data types), give an expression for the number of steps that one needs to move up to get from A to C, in terms of the objects *spotOne* and *spotTwo*. (In terms of the coordinates of the points, this is $Q - Y$.)

(b) Give a specification of a function *CORNER1* that inputs two spots, one at A and one at B, satisfying the condition that B is above and to the right of A, and returns a spot at point C.

(c) Give an implementation of *CORNER1*. Note that we can move a spot from A to C by applying *upOne* the appropriate number of times.

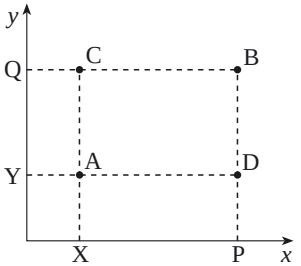


Figure 2.1

(d) Trace *your* implementation in (c) with the inputs *spotOne* in the state $(-1, 3)$ and *spotTwo* in the state (1,7). Verify that the correct value is returned for these inputs.

---

3 Implementing the class Spot

A class is an implemented data type. The specification of a class tells you all that you need to know in order to use it. If one just gave the specification, one would have, in essence, an abstract data type, where the word **abstract** is used to indicate that there is no particular method of implementation incorporated into the data type description. However, a class comes with an implementation built in. If you want to introduce a new class of your own for use in some application, then you need to provide an implementation of the class. This implementation needs to be given in terms of what is already available. At this stage, what we have available are the primitive types described in Unit 3.

3.1 Implementing the attributes and methods

We will now describe how the class **Spot** can be implemented using the primitive types. Recall that we refer to a set giving all possible states of an object of type **Spot** as the **data set** of **Spot**. The implementation of the class **Spot** needs first to describe how the data set of **Spot** is to be implemented. Once this is available, we give an implementation of each of the methods of **Spot**, based on the implementation of its data set.

To implement the data set, we will use the fact that any state of a variable of type **Spot** can be represented by a pair of integers. Call the first of these integers *xPos* and the second of the integers *yPos*. For example, if an object *aSpot* of type **Spot** has state $(5, -3)$, then $xPos = 5$ and $yPos = -3$. Conversely, for a variable of type **Spot**, if we know what the values *xPos* and *yPos* are, then we know what the state of the variable is. So these two values provide a complete description of the data set of the class **Spot**.

We call *xPos* and *yPos* the **attributes** of **Spot**. The first step in implementing a class is to give the attributes that implement its data set. The implementation must state the type of each attribute. This type must be given in terms of what we already have, which is, at present, the primitive data types. To do that is no problem here, since each of *xPos* and *yPos* is an integer, and so can have type **Int**. We give this first step in implementing **Spot** below.

In principle, we need to distinguish between the abstraction (the form of data as discussed in the specification of the class) and its implementation in terms of attributes. In this simple introductory example, however, this distinction is of no great significance!

Attributes
xPos **in Int**
yPos **in Int**

The implementation of each method of a class is given in terms of the attributes. A method will always be applied to a specific instance of the class, which is referred to as the **receiver** object. In code implementing a method, the receiver object is always identified as *this*.

Some programming languages use *this*, others may use some other identifier, such as *self*, to play the same role.

For example, the implementation of the method *getXPos* is given below. The idea of this implementation is simple enough. We just return the value of the attribute *xPos* for the object to which the method is addressed. In the method implementation, this receiver object is identified by *this*, and *this.xPos* identifies the value of the attribute *xPos* for this object.

method *getXPos*() **return in Int**
pre **true**.
post For a spot with state (X, Y), the returned value is X.

{**return** *this.xPos*}

As another example, here is the implementation of *upOne*.

method *upOne*() **state change**
pre **true**.
post Moves the spot upwards on the grid by one position.

$\{this.yPos \leftarrow this.yPos + 1\}$

Again, *this* is the receiver object to which the method is addressed. The desired effect of *upOne* (as given by the postcondition) is to move *this* up one position on the grid. If the state of *this* before *upOne* is applied is (X, Y), then the effect of *upOne* is to increase Y by 1. So the state of *this* after *upOne* is applied is $(X, Y + 1)$.

Now look at the implementation. Remember that *this.yPos* refers to the value of the attribute *yPos* for the receiver object *this*. So the effect of the assignment

$this.yPos \leftarrow this.yPos + 1$

is to change the state of the receiver spot (*this*) by increasing the value of the attribute *yPos* by 1. There is nothing in this code that affects the other attribute *xPos* of this receiver. So the implementation will leave *xPos* unchanged. An increase of 1 in *yPos*, while *xPos* is left unchanged, will move this spot up the grid by one place. So the given implementation of *upOne* will have the correct effect, as given in the postcondition.

We have shown above how, in principle, the methods *upOne* and *getXPos* can be implemented. We shall describe how to enter an implementation of a class into the WorkPad shortly. Until we do, we shall show implementations in principle, and shall include the specification of each method.

Activity 3.1

Give an implementation of the method *getYPos*.

Discussion The implementation is similar to that for *getXPos*. This time you need to return the value of the attribute *yPos* for the receiver object *this*, rather than *xPos*.

method *getYPos*() **return in Int**
pre **true**.
post For a spot with state (X, Y), the returned value is Y.

{**return** *this.yPos*}

Activity 3.2

Give an implementation of the method *rightOne*.

Discussion The implementation is similar to that for *upOne*. This time you need to increase the value of the attribute *xPos* by 1, so as to change the state of the receiver object *this* from (X, Y) to $(X + 1, Y)$.

method *rightOne*() **state change**
pre **true**.
post Moves the spot right on the grid by one position.
$\{this.xPos \leftarrow this.xPos + 1\}$

Activity 3.3

Give implementations of the methods:

(a) *leftOne*;

(b) *downOne*.

Discussion

(a) The effect of *leftOne* is to move the spot to which it is addressed left on the grid by one space. This time, we need to decrease the value of the attribute *xPos* by 1, so as to change the state from (X, Y) to $(X - 1, Y)$.

method *leftOne*() **state change**
pre **true**.
post Moves the spot left on the grid by one position.
$\{this.xPos \leftarrow this.xPos - 1\}$

(b) The effect of *downOne* is to move the spot to which it is addressed down on the grid by one space. We need to decrease the value of the attribute *yPos* by 1, so as to change the state from (X, Y) to $(X, Y - 1)$.

method *downOne*() **state change**
pre **true**.
post Moves the spot down on the grid by one position.
$\{this.yPos \leftarrow this.yPos - 1\}$

When working inside the implementation of a class, we can do two things.

1. We can access the value of each attribute.
2. To set the state of an object, all we need to do is to set the value of each attribute.

When a new object is declared in a **var** statement, it is created in the default state for the class. As part of the implementation of a class, we need to give an implementation of the default state. To do this, we give the values of the class attributes in the default state. For the class **Spot**, this means that we must give the default values of the attributes *xPos* and *yPos*. Now the default state is $(0, 0)$, in which each of the attributes *xPos* and *yPos* is 0. We set a spot to the default state as follows.

Default

$this.xPos \leftarrow 0$
$this.yPos \leftarrow 0$

You might picture the effect of a statement of the form

var *aSpot* **in Spot**

in two parts. First, an area of computer memory suitable to store a variable of type **Spot** is set aside, and the identifier *aSpot* is attached to it. This new object *aSpot* then becomes the receiver for the code details setting the default value. The assignments $this.xPos \leftarrow 0$ and $this.yPos \leftarrow 0$ then set the state of *aSpot* to $(0, 0)$.

3.2 Class implementation in the WorkPad

The course code allows a user to define a new class in the WorkPad. The classes we describe will usually already be implemented in the WorkPad, so you will not need to enter the details of their implementations. But for completeness, we now show you how this can be done. When a class implementation is entered into the WorkPad, we will provide separate identifiers for a method and for the code that implements it. For convenience, we will use a standard naming convention. The implementation of the method *rightOne* for the class **Spot** will be called *rightOneSpot*.

The implementation of **Spot** would be given in the WorkPad as follows.

```
Class Spot
{
this.xPos in Int
this.yPos in Int
this.xPos <-- 0
this.yPos <-- 0
/* These two statements give the implementation of the
 default state. */
this.getXPos calls getXPosSpot
this.getYPos calls getYPosSpot
this.upOne calls upOneSpot
this.downOne calls downOneSpot
this.leftOne calls leftOneSpot
this.rightOne calls rightOneSpot
/* These statements show which code should be used when
 each method of a class is called. */
}
```

```
Spot extends Base
```

```
method   getXPosSpot()
{
return this.xPos
}
```

```
method   getYPosSpot()
{
return this.yPos
}
```

```
method    upOneSpot()
{
this.yPos <-- this.yPos + 1
}
```

```
method    downOneSpot()
{
this.yPos <-- this.yPos - 1
}
```

```
method    leftOneSpot()
{
this.xPos <-- this.xPos - 1
}
```

```
method    rightOneSpot()
{
this.xPos <-- this.xPos + 1
}
```

There are a few things above that are technicalities needed for the WorkPad, and whose effect we shall not explain in detail here. For example, the statement **Spot extends Base** is needed for the WorkPad in order to make *clone* and *show* available for a new class. It tells the WorkPad where to find code to run when *clone* or *show* is called.

When using variables of type **Spot**, all that you need to know is given in the specification of the class **Spot**. Indeed, the way implementations of methods are given inside the class implementation would not form valid code outside the class implementation. Inside the implementation of **Spot**, we can access the attributes *xPos* and *yPos* of the receiver object *this* by writing *this.xPos* and *this.yPos*. Outside the implementation of **Spot**, we cannot work in the same way. Call code written by someone using the class **Spot** (rather than by someone implementing the class) **user code**. In user code, the only methods available to a variable of type **Spot** are those that appear in the specification of **Spot**. We can access the values of the attributes in user code, but only because there are methods available in the specification of **Spot** to do this (*getXPos* and *getYPos*). Some programming languages do allow outside access to attributes, by statements such as

$$mySpot.xPos \leftarrow 0$$

but in the course code, such direct access to attributes is not possible in user code.

This means that you can only change the state of an object of type **Spot** by use of a method of the class **Spot**. You should note that the only methods available in outside code are those that appear in the specification of **Spot**. This **information hiding** of attributes of the class from outside users gives the designer of the class some control over how the class is used, as any outside use of the class is restricted to the methods of the class, as given in the class specification.

Objectives for Section 3

After studying this section you should be able to do the following.

- ❒ Recognise and use the terminology: attribute.
- ❒ Use the notation associated with reading and setting attribute values in the context of a class implementation.
- ❒ Be aware that direct access to attributes is only possible inside the class implementation; user code can only use the methods of the class to manipulate objects.
- ❒ Interpret and use the statements that are used to enter the code implementing a new class into the WorkPad.

Exercises on Section 3

Exercise 3.1

Suppose that the class **Spott** is identical to **Spot** except that it has the default state $(1, -1)$ rather than $(0, 0)$. To declare **Spott** in the WorkPad, how should the code given in the text above for the class **Spot** be modified?

4 Extending the class Spot

In this section, we will consider two further examples of classes. These classes will each be obtained by enhancing the class **Spot**, rather than by starting from scratch. The first example, called **QuickSpot**, enhances the class **Spot** by adding additional methods. The second example, called **ColourSpot**, enhances **Spot** by adding additional structure, a colour, to the spot. Each of **QuickSpot** and **ColourSpot** extends **Spot**. We shall explain the meaning of "extends" later in the section.

4.1 The class QuickSpot

The class **QuickSpot** has the same data set as the class **Spot**, but has additional methods that enable us to move a spot around the grid more quickly. One of these new methods is *upQuick*. Unlike the methods of **Spot**, *upQuick* has a parameter. The effect of the statement $qSpot.upQuick(n)$ is to change the state of *qSpot* by moving it up n places. A specification of *upQuick* is given below.

method *upQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved up by n positions.

We call n a **parameter** of the method *upQuick*. In the method specification, the type of the parameter is given in brackets after the method identifier. The precondition of a method may place conditions on a parameter (as is the case for *upQuick*). For *upQuick*, the parameter n is an integer, and gives the number of places that the receiver is to be moved. The parameter n must be greater than 0.

In general, the signature line of a method will give any parameters required, and their types, in brackets after the method name. Where a method does not require any parameters, then empty brackets are given after the method name. All the methods of **Spot** have signature lines that show that they do not require a parameter.

Suppose that we want to enhance the class **Spot** by adding this method, and other similar methods such as *leftQuick*(n). We will now describe such a class, called **QuickSpot**. This has the same data set as **Spot**, but has additional methods. Changing the type functions gives a different data type (even if the data set is the same). This comment is true for class types, just as for primitive types.

The specification of **QuickSpot** is given below.

QuickSpot **Class specification**

QuickSpot extends Spot

States As for **Spot**.

State notation As for **Spot**.

Default The quickspot is at $(0, 0)$.

Methods
upQuick(n **in Int**) **state change**
downQuick(n **in Int**) **state change**
leftQuick(n **in Int**) **state change**
rightQuick(n **in Int**) **state change**
plus from **Spot**
getXPos, *getYPos*, *upOne*, *downOne*, *leftOne*, *rightOne*

method *upQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved up by n positions.

method *downQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved down by n positions.

method *leftQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved left by n positions.

method *rightQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved right by n positions.

The specification of **QuickSpot** contains the statement

QuickSpot extends Spot

In this course, we regard extension as a relationship between class specifications. To say that **QuickSpot extends Spot** implies that:

1. **QuickSpot** has all the methods of **Spot**. These **inherited methods** are listed in the specification. (To do this is not strictly necessary. It just provides a reminder of what the methods of **Spot** are.)
2. The specifications of the methods inherited from **Spot** are the same as they were for **Spot**.

This is a key property of the notion of extension.

Another key property of extension.

The specifications of inherited methods are not repeated in the specification of **QuickSpot**. (Look back to the specification of **Spot** if you need to see them.)

QuickSpot also has some additional methods, which **Spot** does not have. The specifications of these new methods are given above as part of the specification of **QuickSpot**.

The data set and state notation are the same for **QuickSpot** as they were for **Spot**. Again, these are not repeated here, and you need to look back to the specification of **Spot** if you need to see them.

But in general, a class extending **Spot** may have additional structure.

Activity 4.1

What is the state of *qSpot* after the following code has been executed?

```
var qSpot in QuickSpot
qSpot.downQuick(4)
qSpot.leftQuick(6)
qSpot.upQuick(7)
```

Discussion The default state for **QuickSpot** is $(0, 0)$, so this is the state of *qSpot* after the **var** declaration. After the application of *downQuick*(4), the state is $(0, -4)$, then after *leftQuick*(6), the state is $(-6, -4)$. Finally, after *upQuick*(7), the state of *qSpot* is $(-6, 3)$.

A specification of a function *JUMP* is given below. As far as the effect of this function is concerned, this specification could equally well be given for **QuickSpot** or for **Spot**. The difference lies in what is available when we come to implement *JUMP*.

function *JUMP*(q **in QuickSpot**, m **in Int**, n **in Int**) **return in QuickSpot**
pre **true**.
post If q has the state (X, Y) then the returned value has the state $(X + m, Y + n)$.

For example, *JUMP* applied to a quickspot in the state (2,5), with $m = -4$ and $n = 3$, returns a quickspot in the state $(2 - 4, 5 + 3) = (-2, 8)$.

Activity 4.2

What is the effect of *JUMP* applied to a quickspot in the state $(-3, 1)$, with:

(a) $m = 2$ and $n = 7$;

(b) $m = 5$ and $n = -4$?

Discussion

(a) The returned value is a quickspot in the state $(-3 + 2, 1 + 7) = (-1, 8)$.

(b) The returned value is a quickspot in the state $(-3 + 5, 1 - 4) = (2, -3)$.

Activity 4.3

Give an implementation of *JUMP*.

Discussion Notice that if $n > 0$ and $m > 0$, then the returned value is a quickspot n places on the grid above q and m places on the grid to the right of q. If $n < 0$, then the move is down rather than up, and if $m < 0$, then the move is left rather than right.

Start by initialising a local variable, say *aQSpot*, and set its state to that of the parameter q. If $n > 0$, then we need to move *aQSpot* up. This can be achieved by the statement

if $(n > 0)$ **then** $\{aQSpot.upQuick(n)\}$

If $n < 0$, then the move must be down. This can be achieved by the statement

if $(n < 0)$ **then** $\{aQSpot.downQuick(-n)\}$

Notice that if $n = 0$, then neither of these **if** conditions is true. In that case, neither **then** statement is executed, and so there is no move either up or down (which is as required).

Now, if $m > 0$, a move to the right is needed, which can be achieved by the statement

if $(m > 0)$ **then** $\{aQSpot.rightQuick(m)\}$

If $m < 0$, this move must be left. This can be achieved by the statement

if $(m < 0)$ **then** $\{aQSpot.leftQuick(-m)\}$

An implementation of *JUMP* is given below. (The order in which the four conditional statements are given does not matter.)

```
function JUMP(q, m, n)
{
var aQSpot in QuickSpot
aQSpot <-- q.clone()
if (n > 0) then {aQSpot.upQuick(n)}
if (n < 0) then {aQSpot.downQuick(-n)}
if (m > 0) then {aQSpot.rightQuick(m)}
if (m < 0) then {aQSpot.leftQuick(-m)}
return aQSpot
}
```

4.2 Implementing the class QuickSpot

Later in Section 4 we examine how the class **QuickSpot** can be implemented in the WorkPad. We look first at how we can implement **QuickSpot** in principle. Much of the work needed to implement **QuickSpot** is done automatically by saying that **QuickSpot extends Spot**. The class **QuickSpot** inherits the attributes of **Spot** (which are *xPos* and *yPos*). For the class **QuickSpot**, these inherited attributes give a complete description of the data set. **QuickSpot** also inherits a number of its methods, such as *upOne*, from **Spot**. What will be needed for the implementation of **QuickSpot** are implementations of the new methods, such as *upQuick*.

Now we know that the code below will move the spot *aSpot* up n places.

```
var index in Int
for (index <-- 1 to n)
    {aSpot.upOne()}
```

To use code of this form to implement *upQuick*, we must show that the spot to be moved (*aSpot* in this code) is the object to which the method is addressed (the receiver). We can implement *upQuick* as below.

method *upQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved up by n positions.

```
{
var index in Int
for (index ← 1 to n)
    {this.upOne()}
}
```

Remember, from page 24, that we use *this* as the identifier of the receiver object.

The implementations of the methods of **Spot** all used direct reference to the attributes. Here, we are working in a different way. Because **QuickSpot** extends **Spot**, an object of type **QuickSpot** can respond to the methods of type **Spot**, and so can be treated as an object of type **Spot**. When implementing a method in this way, we again use *this* as the identifier of the receiver object.

Activity 4.4

Give an implementation of the method *leftQuick*.

Discussion We can use a similar approach, as below.

method *leftQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved left by n positions.

{
var *index* **in Int**
for ($index \leftarrow 1$ **to** n)
{$this.leftOne()$}
}

The methods *rightQuick* and *downQuick* can be implemented in a similar way.

This is not the only way that we could implement the methods of **QuickSpot**. Because **QuickSpot** is an extension of **Spot**, the attributes of **Spot**, *xPos* and *yPos*, are also attributes of **QuickSpot**. Consequently, it is also possible to give implementations of the methods of **QuickSpot** in terms of these attributes. We do not need to do this, of course, since we have seen how to implement the methods of **QuickSpot**. However, it is useful to see how it can be done. Essentially, we would be working in the same sort of way as we did in Section 3, when giving implementations of the methods of **Spot**.

For example, let us see how we can give an implementation of the method *rightQuick* in terms of the attributes *xPos* and *yPos*. The effect of *rightQuick* is to move a quickspot right n places. So if the quickspot is in the state (X, Y), an application of *rightQuick*(n) will move it to $(X + n, Y)$. Thus we need to change the attribute *xPos* of the object addressed by *rightQuick*, but *yPos* does not need to change. We can write an implementation based on this idea as below.

method *rightQuick*(n **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved right by n positions.

{$this.xPos \leftarrow this.xPos + n$}

In this case, implementation using the attributes is even more straightforward than using methods of **Spot**. (In general, it is up to the designer of a new class to decide which approach to use to the implementation of a method.)

Activity 4.5

Using the attributes *xPos* and *yPos*, give an implementation of the method *downQuick* of **QuickSpot**.

Discussion Application of the method $downQuick(n)$ moves an object from (X, Y) to $(X, Y - n)$. An implementation is given below.

method *downQuick*(*n* **in Int**) **state change**
pre $n > 0$.
post The quickspot is moved down by n positions.
$\{this.yPos \leftarrow this.yPos - n\}$

As with **Spot**, note that direct access to the attributes *xPos* and *yPos* is only possible *within* the implementation of the class **QuickSpot**. In user code, objects of type **QuickSpot** can only respond to methods declared in the class specification. In user code, we can access the attribute values by using the methods *getXPos* and *getYPos*, but we cannot set the state of a quickspot by setting the values of its attributes.

Some classes will have methods that set the state of attributes.

4.3 The class ColourSpot

Our second example of extension is a class that enhances **Spot** by adding extra structure. A coloured spot is a spot that has a colour. The methods for moving a coloured spot around the grid are identical to the methods of the class **Spot**. The class **ColourSpot** has two additional methods: *nextColour*, which allows you to change the colour of a coloured spot; and *getColour*, which returns an integer that represents the colour of a coloured spot.

The specification of the class **ColourSpot** is given below.

ColourSpot **Class specification**

ColourSpot extends Spot

States The state of an object is a position on a two-dimensional square grid (as in **Spot**) together with a colour.

State notation (X, Y, C) means that the spot has position (X, Y) and colour C. The possible colours are: Black, Red, Green and Blue.

Default (0, 0, Black)

Methods
getColour() **return in Int**
nextColour() **state change**
plus from **Spot**
getXPos, *getYPos*, *upOne*, *downOne*, *leftOne*, *rightOne*.

method *getColour*() **return in Int**
pre **true**.
post The returned value represents the colour of the coloured spot. If the spot is Black, then 0 is returned; if the spot is Red, 1 is returned; if the spot is Green, 2 is returned; if the spot is Blue, 3 is returned.

method *nextColour*() **state change**
pre **true**.
post The colour of the spot is changed, according to the following order: Black is changed to Red, Red is changed to Green, Green is changed to Blue, and Blue is changed to Black. The position of the spot is not changed.

The methods inherited from **Spot** do not change the colour of a coloured spot.

Activity 4.6

What is the state of *colSpot* after the following code has been executed?

```
var colSpot in ColourSpot
colSpot.nextColour()
colSpot.nextColour()
colSpot.rightOne()
colSpot.rightOne()
colSpot.downOne()
```

Discussion After the **var** declaration, the state of *colSpot* is the default: (0, 0, Black). The subsequent statements change the state as follows:

(0, 0, Red); (0, 0, Green); (1, 0, Green); (2, 0, Green); $(2, -1,$ Green).

Activity 4.7

Give an implementation of the function *SPOTCOL*, specified below.

function *SPOTCOL*(n **in Int**) **return in ColourSpot**
pre n is 0, 1, 2 or 3.
post The returned value is a coloured spot whose state is $(0, 0,$ colour), where colour is Black if $n = 0$, Red if $n = 1$, Green if $n = 2$ and Blue if $n = 3$.

Discussion Starting with a coloured spot in the default state, we need to apply *nextColour* n times. An implementation is given below.

```
function SPOTCOL(n)
{
var cSpot in ColourSpot
var index in Int
for (index <-- 1 to n)
  {cSpot.nextColour()}
return cSpot
}
```

We now look at how we can implement the class **ColourSpot**. For the class **QuickSpot**, the data set is identical to that of **Spot**. For **ColourSpot**,

the data set is "a spot and more". In the implementation of **ColourSpot**, this view is reflected in the attributes. **ColourSpot** inherits the attributes of **Spot**. In addition, **ColourSpot** has another attribute, giving the colour of the coloured spot.

So **ColourSpot** has three attributes:

xPos **in Int**
yPos **in Int**
colour

The first two of these attributes are inherited from **Spot**. The third is not yet properly described. We need to choose a data type to implement the attribute *colour*. It is convenient to represent the colour as an integer. (This form of representation is suggested in the specification of *getColour*.) We will use:

0 to represent Black
1 to represent Red
2 to represent Green
3 to represent Blue

So, in the implementation, the state with $xPos = 4$, $yPos = -2$, $colour = 2$, will represent a coloured spot with position $(4, -2)$ and with colour Green.

The attributes in the implementation of **ColourSpot** are given below.

Attributes
xPos **in Int**
yPos **in Int**
/* from **Spot** */
colour **in Int**
/* 0 means Black, 1 means Red, 2 means Green, 3 means Blue. */

Notice that:

ColourSpot has all the attributes of **Spot**.

This is another general feature of extension of classes.

In extension of classes, the situation for attributes is like that for methods. If **A extends B**, then the class **A** has all the methods of the class **B**, and the class **A** also has all the attributes of the class **B**. Class **A** may have additional methods that **B** does not have (as illustrated by **QuickSpot extends Spot**). And class **A** may have additional attributes that **B** does not have (as illustrated by **ColourSpot extends Spot**). In general, class **A** may have both additional attributes and additional methods. Indeed, this is the case for **ColourSpot**, which has two methods, *getColour* and *nextColour*, that are not methods of **Spot**.

When writing implementations of the methods of the class **Spot**, we had access to the attributes of **Spot**, *xPos* and *yPos*. Similarly, when writing implementations for the methods of **ColourSpot**, we can access the new attribute, *colour*, of **ColourSpot**, as well as *xPos* and *yPos*. We only need to provide implementations for the new methods (those not inherited from **Spot**), which are *getColour* and *nextColour*.

EXAMPLE 4.1

An implementation of *nextColour* that could be used in the implementation of the class **ColourSpot** is given below. This implementation needs to change the attribute *colour* in the following way: 0 is changed to 1; 1 to 2; 2 to 3, and 3 to 0.

method *nextColour*() **state change**
pre **true**.
post The colour of the spot is changed, according to the following order: Black is changed to Red, Red is changed to Green, Green is changed to Blue and Blue is changed to Black. The position of the spot is not changed.

{
if $(this.colour == 3)$ **then** $\{this.colour \leftarrow 0\}$
else $\{this.colour \leftarrow this.colour + 1\}$
}

Alternatively the single statement $this.colour \leftarrow MOD(this.colour + 1, 4)$ gives the required result.

The method *getColour* is straightforward to implement. We just need to return the value of the attribute *colour*.

Activity 4.8

Give an implementation of *getColour* that could be used in the implementation of the class **ColourSpot**.

Discussion An implementation is given below.

method *getColour*() **return in Int**
pre **true**.
post The returned value represents the colour of the coloured spot.

{**return** *this.colour*}

To give a complete implementation of **ColourSpot**, we also need to give an implementation of the default state. For the inherited attributes, *xPos* and *yPos*, the corresponding default values are automatically inherited. So we only need to give code for the default value of the new attribute, *colour*. The default colour is Black, which is represented by 0. So the statement $this.colour \leftarrow 0$ will need to be included.

4.4 Implementation of QuickSpot and ColourSpot in the WorkPad

You saw in Section 3.2 how the class **Spot** can be implemented in the WorkPad. We look now at how this can be done for the classes **QuickSpot** and **ColourSpot**. We look first at **QuickSpot**. (As for **Spot**, the classes **QuickSpot** and **ColourSpot** are already implemented in the WorkPad. So you do not need to enter any code to work with variables of these types in the WorkPad. What we show here is how these classes could be implemented in the WorkPad if only the primitive types and the class **Spot** were available.)

Much effort is saved by the decision to say that **QuickSpot extends Spot**. This means that **QuickSpot** will inherit all the features of **Spot**. For **Spot**, we first gave the default values of the attributes. For **QuickSpot**, the default values of the attributes are identical to those of **Spot**, and these are automatically inherited. So no instructions setting default attribute values are needed for **QuickSpot**. The methods *getXPos*, *getYPos*, *upOne*, *downOne*, *leftOne* and *rightOne* are all inherited from **Spot**, and their implementations are again automatically inherited from **Spot**. There is no need to include any code statement about that. We do need to attach an identifier to the code implementing each of the new methods, and to include a code statement saying that each method calls the appropriate implementation. We will adopt the same sort of naming convention as we did for **Spot**. The implementation of *upQuick* will be called *upQuickQuickSpot* (and so on).

We also need to give the code for each of the method implementations. Earlier, we discussed two possible approaches to implementing the methods of **QuickSpot**. To illustrate both approaches, we have used one approach for two of the methods, and the alternative approach for the other two. (In practice, one would be more likely to use the same approach in all cases.)

```
Class QuickSpot
{
/* Default state is inherited from Spot. */
this.upQuick calls upQuickQuickSpot
this.downQuick calls downQuickQuickSpot
this.leftQuick calls leftQuickQuickSpot
this.rightQuick calls rightQuickQuickSpot
/* The implementations of getXPos, getYPos, upOne, downOne,
 leftOne and rightOne are all inherited from Spot. */
}

QuickSpot extends Spot
```

```
method  upQuickQuickSpot(n)
{
var index in Int
for (index <-- 1 to n)
 {this.upOne()}
}
```

```
method  downQuickQuickSpot(n)
{
this.yPos <-- this.yPos - n
}
```

```
method  leftQuickQuickSpot(n)
{
var index in Int
for (index <-- 1 to n)
  {this.leftOne()}
}
```

```
method  rightQuickQuickSpot(n)
{
this.xPos <-- this.xPos + n
}
```

All the details necessary for the implementation of **QuickSpot** are given above. Anything that appears to be missing, such as an implementation of *upOne*, is inherited from **Spot**. (Code for *clone* is inherited from **Base** via **Spot**.)

The code that would be used to declare **ColourSpot** as a new class in the WorkPad is given below.

```
Class ColourSpot
{
this.colour in Int
this.colour <-- 0
/* Gives the default value of the attribute colour.
Default values for xPos and yPos are inherited from Spot. */
this.getColour calls getColourColourSpot
this.nextColour calls nextColourColourSpot
/* The implementations of getXPos, getYPos, upOne, downOne,
leftOne and rightOne are all inherited from Spot. */
}
ColourSpot extends Spot
```

```
method   getColourColourSpot()
{
return this.colour
}
```

```
method   nextColourColourSpot()
{
if (this.colour == 3) then {this.colour <-- 0}
else {this.colour <-- this.colour + 1}
}
```

4.5 Aside on extension and type

Read Only
The material in this section is not assessed directly.

The comments in this subsection are here to help you to relate your work in this course to what you may encounter elsewhere. Do not expect to understand them at a first reading.

In the course code, saying that **A extends B** has the following implications about the classes **A** and **B**.

Class **A** has all the methods of class **B**.
Class **A** has all the attributes of class **B**.

Class **A** may have additional features, of course, as well as those it has inherited from class **B**. These additional features may be extra methods, or additional attributes, or both.

For example, an object of type **ColourSpot** can respond to all the methods of the class **Spot**. A coloured spot can show all the behaviour associated with a spot (and more). We say that the data type **ColourSpot** is a **subtype** of the data type **Spot**. (The use of the prefix "sub" here is a reflection of the idea that every coloured spot is a spot; so the collection of coloured spots is contained within the collection of spots.)

To say that **X** is a **subtype** of **Y** has the following implications.

1. A variable of type **X** can be recognised as being of type **Y**.
2. All methods of type **Y** are available to a variable of type **X**.
3. The effect of such a method in type **X** is consistent with its effect in type **Y**.

In the course code, the **extends** relationship is used in a way that ensures subtyping. That is, if **A extends B**, then class **A** is a subtype of class **B**.

Now the idea of subtype is a relationship between class *specifications*. This relationship between specifications is usually achieved through **inheritance**. Inheritance is a relationship between class implementations. If a class **A** inherits from class **B**, then the key implication relates to the way the implementation of a method is found. Suppose that class **A** inherits from class **B**, and that the specification of **A** includes a method *aMethod*, and *aMethod* is called in code.

If a class **A** inherits from class **B** we say that **B** is a **parent** class of **A**, and **A** is a **child** class of **B**.

1. First, look for an implementation of *aMethod* within class **A**. If there is one, then that implementation is used.
2. If there is no implementation of *aMethod* in class **A**, then look for an implementation of *aMethod* in class **B**, and use that.

There must either be an implementation of *aMethod* in class **A**, or one available through class **B**. It is incumbent on whoever implements class **A** to ensure that. However, the process can be extended. We can have: class **A** inherits from class **B**, and class **B** inherits from class **C**, and class **C** inherits from class **D** (and so on). In such a case, we keep looking back until we find an implementation of *aMethod*, and we use the first implementation that we come to. (So: look first in **A**, then in **B**, then in **C**, then in **D**, and so on. Stop when you find an implementation of *aMethod*.)

Inheritance and subtyping are separate issues, in that subtyping is about behaviour (which methods are available to objects of that type and what are their specifications), while inheritance is about how it is achieved (how, and in which class, the behaviour is implemented). One can say the same about type (class specification) and class (which includes the implementation). Type describes the behaviour, class includes the way the behaviour is achieved.

Object-oriented programming languages make considerable use of inheritance. Do not expect extension as described here to be the same as inheritance in a programming language. Inheritance is usually much more general. In particular, extension in this course does not allow inherited methods to be overridden; that is, the specification of an inherited method may not be changed.

Also, the view of class that we use in this course is a simplified one.

Object-oriented programming languages may contain statements to support types separately from an implementation. Doing this leads to additional complications that we do not wish to address in this course.

For example, Java uses the idea of **interface** to do this.

Another issue that we will not discuss is the use of overloaded method identifiers in classes, that is, where the same method identifier is used in several classes. The combined use of overloaded identifiers and class extension leads to complications that again we do not wish to address at this level.

Objectives for Section 4

After studying this section you should be able to do the following.

- ❒ Recognise and use the terminology **extends** in the context of a class specification.
- ❒ Interpret a class specification involving extension.
- ❒ Interpret the specifications of **QuickSpot** and **ColourSpot**.
- ❒ Trace code involving objects of the types **QuickSpot** and **ColourSpot**.
- ❒ Give an implementation of a method for a class whose specification involves extension.

Exercises on Section 4

Exercise 4.1

What is the state of *qSpot* after the code below has been executed?

```
var qSpot in QuickSpot
qSpot.upQuick(3)
qSpot.leftOne()
qSpot.leftQuick(1)
qSpot.downQuick(2)
```

Exercise 4.2

What is the state of *colSpot* after the code below has been executed?

```
var colSpot in ColourSpot
var index in Int
colSpot.nextColour()
for (index <-- 1 to 4)
  {colSpot.upOne()}
```

Exercise 4.3

This question concerns an object *anObj* whose type is one of: (i) **ColourSpot**; (ii) **Spot**; (iii) **QuickSpot**.

(a) For which of the types (i)–(iii) is the statement $anObj.upOne()$ valid?

(b) For which of the types (i)–(iii) is the statement $anObj.upQuick(1)$ valid?

Exercise 4.4

Give an implementation of the function F, specified below.

function $F(n$ **in Int**$)$ **return in ColourSpot**
pre $n > 0$.
post The returned value is a coloured spot in the state $(n, -n,$ Green$)$.

Summary of Unit 4

1. We introduced new language associated with a class type: **object** for a variable of class type, and **method** for a process associated with a class. You met the dot notation for application of a method.

   The possible states of an object of type **Spot** are positions on a two-dimensional grid. A possible state is represented by a pair of integers, (X, Y). **Spot** has a method that returns X and a method that returns Y. It also has methods that change the state of an object by moving it one place on the grid, either up, down, left or right.

   The specification of the class **Spot** gives the following information, which is all that is needed when working with code involving objects of type **Spot**.

   The possible states of an instance of the class.
   The default state.
   The identifiers of the methods.
   A specification of each method.
2. We looked at code involving variables of type **Spot**. We can specify a user-defined function with class types in the signature, and you saw examples of functions involving the class **Spot**.

   For variables a and b of class type, we copy the state of b to a using a statement $a \leftarrow b.clone()$. This means: "change the state of a so that it becomes identical to the current state of b". (Using *clone* ensures that the link between a and b is temporary. Any subsequent changes to the state of b do not change the state of a.)
3. You saw how to implement the class **Spot**. The first step is to describe its possible states. This is done using attributes. **Spot** has two attributes: *xPos* in **Int** and *yPos* in **Int**. For a spot in the state (X, Y), X gives the value of the attribute *xPos* and Y gives the value of the attribute *yPos*. Implementations of the methods and of the default state of **Spot** were then given using these attributes. The approach of directly accessing the attributes is valid code only inside the implementation. In user code involving objects of type **Spot**, you cannot set the state of an object by setting the values of the attributes. Such user code must only use methods given in the specification of **Spot**.
4. If **A extends B**, then class **A** has all the methods of class **B**, and **A** has all the attributes of **B**. Class **A** may have either or both of: more methods or more attributes.

   You saw an example (**QuickSpot extends Spot**) with more methods, and an example (**ColourSpot extends Spot**) with more attributes (and more methods).

Solutions to the exercises

Solution 1.1

The state is $(-3, 2)$. Starting from an initial state of $(-3, 2)$, *downOne* will change the state to $(-3, 1)$ and *rightOne* will then change the state to $(-2, 1)$. Finally, the second application of *rightOne* will change the state to $(-1, 1)$.

Solution 1.2

A trace is given below. In the first **for** loop, *count* has value 1, so in the trace the loop begins and ends on the same line.

Comment	*index*	*count*	*spotOne*
var *spotOne* in **Spot**	-	-	$(0, 0)$
$count \leftarrow 1$	-	1	$(0, 0)$
first loop - loop end	1	1	$(0, 1)$
$count \leftarrow count + 1$	-	2	$(0, 1)$
second loop	1	2	$(1, 1)$
loop end	2	2	$(2, 1)$
$count \leftarrow count + 1$	-	3	$(2, 1)$
third loop	1	3	$(2, 0)$
	2	3	$(2, -1)$
loop end	3	3	$(2, -2)$

Solution 2.1

(a) The fragment below has the required effect.

var *index* **in Int**
for ($index \leftarrow 1$ **to** n)
    {*aSpot.downOne*()}

(b) A specification and implementation are given below.

function *GODOWN*(*s* **in Spot**, *n* **in Int**) **return in Spot**
pre $n \geq 0$.
post The returned value is a spot n places below s.

```
function GODOWN(s,n)
{
var index in Int
var aSpot in Spot
aSpot <-- s.clone()
for (index <-- 1 to n)
  {aSpot.downOne()}
return aSpot
}
```

Solution 2.2

(a) An implementation is given below.

```
function U1R2(s)
{
var temp in Spot
temp <-- s.clone()
temp.upOne()
temp.rightOne()
temp.rightOne()
return temp
}
```

(b) A trace is given below.

	spotOne	*spotTwo*	*spotThree*
var *spotOne* **in Spot**	$(0,0)$		
var *spotTwo* **in Spot**	$(0,0)$	$(0,0)$	
var *spotThree* **in Spot**	$(0,0)$	$(0,0)$	$(0,0)$
$spotTwo \leftarrow U1R2(spotOne)$	$(0,0)$	$(2,1)$	$(0,0)$
$spotThree \leftarrow U1R2(spotTwo)$	$(0,0)$	$(2,1)$	$(4,2)$

Solution 2.3

(a) We can use *getYPos* to find the second coordinate of a spot.

$$spotTwo.getYPos() - spotOne.getYPos()$$

gives the required number of steps.

(b) A specification is given below.

function *CORNER1*(*spotOne* **in Spot**, *spotTwo* **in Spot**) **return in Spot**
pre *spotTwo* is above and to the right of *spotOne*.
post If *spotOne* has state (X, Y) and *spotTwo* has state (P, Q), then the returned value is a spot with state (X, Q).

(c) An implementation is given below.

```
function CORNER1(spotOne, spotTwo)
{
var count in Int
var index in Int
var aSpot in Spot
aSpot <-- spotOne.clone()
/* count gives the number of places to move up from spotOne. */
count <-- spotTwo.getYPos() - spotOne.getYPos()
/* The precondition ensures that count > 0. */
for (index <-- 1 to count)
  {aSpot.upOne()}
return aSpot
}
```

(d) A trace is given below. The value returned is $(-1, 7)$. The postcondition states that the returned value should be a spot whose first coordinate is the same as that of *spotOne*, and whose second coordinate is the same as that of *spotTwo*. So the -1 comes from $(-1, 3)$ (the input value of *spotOne*) and the 7 comes from (1,7) (the input value of *spotTwo*), and the correct value has been returned for these inputs.

Inputs: *spotOne* with value $(-1, 3)$, *spotTwo* with value $(1, 7)$			
Comment	*index*	*aSpot*	*count*
var *aSpot* **in Spot**	-	$(0, 0)$	-
$aSpot \leftarrow spotOne.clone()$	-	$(-1, 3)$	-
$count \leftarrow spotTwo.getYPos() - spotOne.getYPos()$	-	$(-1, 3)$	$7 - 3 = 4$
loop	1	$(-1, 4)$	4
	2	$(-1, 5)$	4
	3	$(-1, 6)$	4
loop end	4	$(-1, 7)$	4
return *aSpot*	-	$(-1, 7)$	4

A spot with value $(-1, 7)$ is returned.

Solution 3.1

We need to change the first statement to read

```
Class Spott
```

and a later statement to read

```
Spott extends Base
```

Also, we need to change the two lines of code giving the default values. For **Spott**, they become

```
this.xPos <-- 1
this.yPos <-- -1
```

Solution 4.1

The state of *qSpot* is initially the default state, $(0, 0)$. The statements move *qSpot* to: (0,3), then $(-1, 3)$, then $(-2, 3)$, then $(-2, 1)$. This is the required final state. (Notice that *leftQuick*(1) and *leftOne* have identical effects.)

Solution 4.2

The state of *colSpot* is initially the default state (0, 0, Black). The first statement takes this to (0, 0, Red). The **for** loop moves *colSpot* up 4 places, to (0, 4, Red). This is the required final state.

Solution 4.3

(a) This statement is valid in all three cases.

(i) *upOne* is a method of the class **ColourSpot** (inherited from **Spot**).

(ii) *upOne* is a method of the class **Spot**.

(iii) *upOne* is a method of the class **QuickSpot** (inherited from **Spot**).

(b) This statement is valid only for (iii). The parametrised method *upQuick* is a method only of the class **QuickSpot**. *upQuick* is not a method of the class **Spot**, nor is it a method of **ColourSpot**. (It is neither inherited from **Spot**, nor is it one of the new methods given in the specification of **ColourSpot**.)

Solution 4.4

Starting with a coloured spot in the default state, we need to:

move n places right;
move n places down;
apply *nextColour* twice (Black to Red then Red to Green).

As it happens, the order in which we do these three things does not matter; so you may well have them in some other order (which will still be correct). We can move n places right and n places down using a single **for** loop. An implementation is given below.

```
function F(n)
{
var colsp in ColourSpot
var index in Int
/* The state of colsp is the default, (0,0,Black). */
for (index <-- 1 to n)
{
  colsp.rightOne()
  colsp.downOne()
}
/* The state of colsp is now (n,-n,Black). */
colsp.nextColour()
/* The state of colsp is now (n,-n,Red). */
colsp.nextColour()
/* The state of colsp is now (n,-n,Green). */
return colsp
}
```

Index